Unit 3

Syntax and Semantics Lexical and Syntax Analysis

Syntax and Semantics

- Syntax
- Definition:
- Syntax refers to the form of its expression or structure of a program — how the elements (tokens) of the language must be arranged.

- Purpose:
- To define valid sentences (statements, declarations) in a programming language.

- Specified By:

- Lexical rules → define tokens (keywords, identifiers, operators, literals, etc.).

- Grammatical rules → define how tokens combine into valid constructs.

- Formal description of syntax of programming languages , for simplicity’s sake do not include description of lowest level syntactic units.
- This small units are called lexemes. The description of lexemes can be given by a lexical specification, which is separate from syntactic descriptionof language.
- The lexemes of programming language include its numeric literals,operators, and special words, among others.
- It can be think of programs as strings of lexemes rather than of characters.

- Lexemes are partitioned into groups-for example, the names of variables, methods, classes, and so forth in a programming language form a group called identifiers.
- Each lexeme group is represented by a name, or token. So, a token of a language is a category of its lexemes.
- For example, an identifier is a token that can have lexemes, or instances, such as sum and total.
- In some cases, a token has only a single possible lexeme.

- For example,
- the token for the arithmetic operator symbol + has just one possible lexeme.
- Consider the following Java statement:
- index = 2 *count + 17;
- Write down lexemes and tokens of this statement :
- The lexemes and tokens of this statement are
- **Lexemes** **Tokens**
- Index identifier
- = equal_sign
- 2 int_literal

- * Mult_op
- Count identifier
- + plus_op
- 17 int_literal
- ; semicolon

Formal Methods of Describing Syntax

- Formal Representation:
- Formal language generation mechanism , usually called as grammar, that commonly used to describe the syntax of programming languages.

- Usually defined using Backus–Naur Form (BNF) or Context-Free Grammars (CFG).

- Example:

- if <boolean_expr> then <statement>

- Here <boolean_expr> and <statement> are syntactic categories.

- In programming language syntax, *syntactic categories* are the classes of language constructs that are defined by the grammar and used to group similar syntactic elements together.
- Think of them as "types of phrases" in the language's grammar — much like nouns, verbs, and adjectives in natural languages.
- **Def:** A syntactic category is a set of strings (sequences of tokens) that are recognized by a single nonterminal symbol in the language's grammar.
- In formal grammar terms: each syntactic category corresponds to a nonterminal in the grammar rules.
- In practical terms: they describe *what kinds of components* are allowed in a certain position in the program.

- In programming, grammar is the set of formal rules that describe the syntax of a programming language.
- It tells us:

- What sequences of symbols (keywords, identifiers, operators, etc.) are valid programs.

- How to structure statements correctly.
- Just like English grammar governs sentences, programming grammar governs programs.

- Formal Definition
-
- A grammar G is defined as a 4-tuple:
-
- G = (N, T, P, S)
-
- where:
-
- N → Non-terminals (syntactic variables like <expr>, <stmt>)
-
- T → Terminals (tokens like if, +, id, ;)
-
- P → Productions (rules that define how terminals & non-terminals combine)
-
- S → Start symbol (usually <program>)

- Let’s define a small grammar for arithmetic expressions:

- 1. <expr> → <expr> + <term> | <expr> - <term> | <term>
- 2. <term> → <term> * <factor> | <term> / <factor> | <factor>
- 3. <factor> → (<expr>) | id | number

- **This grammar says:**

- An expression can be built from terms connected with + or -.

- A term can be built from factors connected with * or /.

- A factor can be an identifier (id), a number, or another expression inside parentheses.
- So the string:
- id + number * id
- is valid according to the grammar.

- Types of Grammars (Chomsky Hierarchy)
-
- Programming languages mostly use Context-Free Grammar (CFG).
-
- 1. Regular Grammar → Used for tokens (via lexical analysis). Example: identifiers, keywords, numbers (handled by regex).

- 2. Context-Free Grammar (CFG) → Used for syntax (via parsing). Example: if-else, while, expressions.
-

Role of Grammar in Compiler Design

- 1. Lexical Analysis → breaks code into tokens (with regex).
- 2. Syntax Analysis (Parsing) → checks if sequence of tokens follows grammar rules.

- Builds a parse tree.
- 3. Semantic Analysis → ensures meaning (e.g., type checking).

- Example in Programming (If Statement Grammar)
-
- Grammar for an if-statement:
-
- <stmt> → if (<expr>) <stmt> else <stmt>
- | if (<expr>) <stmt>
- | <other>

- **Valid:**
- if (x > 0)
- y = 1;
- else
- y = -1;

Invalid: if x > 0 y = 1 else y = -1;

- (because it violates parentheses and structure rules)

- Grammar in programming = formal rules defining valid syntax.

- Mostly defined as context-free grammars (CFGs).
-
- Crucial for parsing & compiler design.
-
- Without grammar, we cannot write compilers or interpreters.
-

Examples in Programming Languages

Syntactic Category	Meaning	Example Elements
Expression	Represents a computation producing a value	a + b, x * (y - z)
Statement	Represents an action to be performed	if (x > 0) {...}, while (true) {...}
Declaration	Introduces variables, functions, classes	int x;, function foo() {...}
Program	The complete valid sequence of constructs forming a program	The whole .java or .c file

Example in BNF (Backus–Naur Form)

```
<expression> ::= <identifier>
               | <expression> "+" <expression>
               | <expression> "*" <expression>

<statement> ::= "if" "(" <expression> ")" <statement>
              | "while" "(" <expression> ")" <statement>
              | "{" <statement_list> "}"

<statement_list> ::= <statement>
                   | <statement> <statement_list>
```

Here:

<expression>, <statement>, and <statement_list> are syntactic categories.

They group together valid constructs of the same kind.

Key points:

- Tokens (like keywords, operators, identifiers) are the smallest units.
- Syntactic categories group tokens and smaller constructs into higher-level constructs.
- They are defined recursively in the grammar rules.
- Parsing uses syntactic categories to build a parse tree or syntax tree.

Semantics

- Definition:
- Semantics refers to the meaning of a syntactically correct program.

- **Types of Semantics:**

- Static semantics – rules checked before runtime (e.g., type checking, declaration-before-use).

- Dynamic semantics – meaning of programs during execution (behavior produced by execution).

- **Formal Approaches:**

- Operational semantics – describes meaning via execution steps on an abstract machine.

- Denotational semantics – maps language constructs to mathematical objects/functions.

- Axiomatic semantics – uses logic to describe behavior (preconditions, postconditions).

A. Static Semantics

- Definition: Rules that can be checked at compile time before the program runs.
- Purpose: Ensure certain constraints are met so that errors are avoided at runtime.
- Examples:
- Type checking:

- int x;
- x = "hello";
- // type error detected before running

- Declaration-before-use:

- y = 5;
- // 'y' not declared before use

- These checks are part of semantic analysis in compilers.

B. Dynamic Semantics

- Definition: The meaning of a program in terms of its runtime behavior — how it behaves when executed.
- Examples:
- Division by zero:
- x = 5 / 0 # runtime error
- Even if syntax is fine, the meaning at runtime leads to an error.
- Output behavior:
- print("Hello") # prints Hello during execution

Formal Approaches to Describing Semantics

- To precisely describe a programming language's meaning, we use **formal semantics** approaches.
  These approaches are **mathematical/logical models**.
- **A. Operational Semantics**

- Idea: Describe meaning by simulating execution on an abstract machine step-by-step.

- Analogy: Like a cooking recipe → follow each step exactly.

- Example (simple arithmetic expression):

- Let’s define an abstract machine rule:

- $\langle x + 1, \{x = 3\} \rangle \Rightarrow 4$

- Means: In a state where x is 3, the expression x + 1 evaluates in one step to 4.

- **Use case: Language interpreters are often described this way.**

B. Denotational Semantics

- Idea: Map each language construct to a mathematical object (often a function).
- Analogy: Like mapping a sentence to its mathematical meaning, ignoring how it's executed.
- Example:
- The meaning of an arithmetic expression is a function from variable states to numbers.

- Let E[x+1] = λσ. σ(x) + 1
- Where:

- σ is the state (mapping variables to values)

- If σ(x) = 3, then (λσ. σ(x) + 1)(σ) = 4

- **Use case: Compiler correctness proofs, mathematical analysis.**

C. Axiomatic Semantics

- Idea: Use logic (preconditions & postconditions) to describe program behavior.

- Analogy: Like proving in math that if this is true before, that will be true after.

- Example (Hoare triples):
- {x = 3} x := x + 1 {x = 4}
- Read as:
- Precondition: x = 3 before statement
- Postcondition: x = 4 after statement

- Example in loop verification:

- {n > 0} factorial := 1; i := 1;
- while i <= n do
- factorial := factorial * i;
- i := i + 1
- end
- {factorial = n!}

- **Use case: Proving correctness of programs (formal verification).**

Comparison Table

Aspect	Operational	Denotational	Axiomatic
What it shows	Step-by-step execution	Mathematical meaning	Logical properties
Good for	Implementing interpreters	Reasoning about meaning	Proving correctness
Example Output	State transitions	Functions mapping states	Proof obligations

2. Lexical and Syntax Analysis

2.1 Lexical Analysis

- Purpose:
- First phase of compilation; converts a sequence of characters into tokens.

- Component:
- Lexical analyzer (scanner).

- Responsibilities:

- Remove whitespace and comments.

- Group characters into lexemes and assign them a token type.

- Pass tokens to the parser.

- Example:
- total = total + sum;

- Tokens:

- <identifier,total> <operator,=> <identifier,total> <operator,+> <identifier,sum> <separator,;>

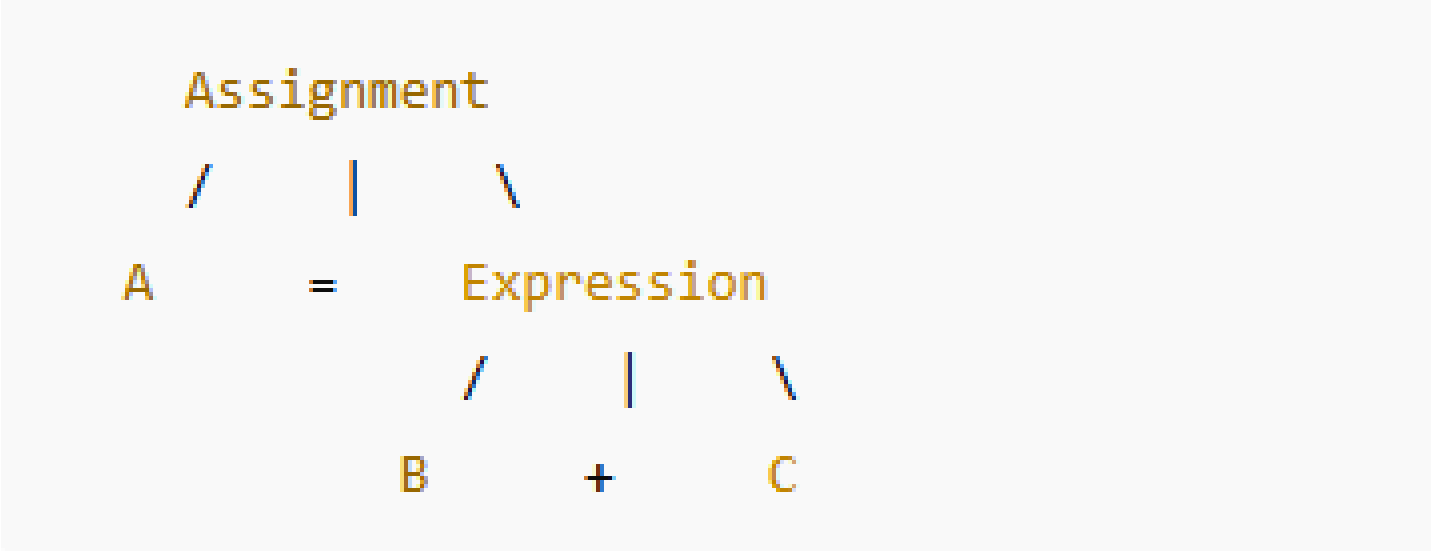
- Tools:
- Lexical analyzers often generated by Lex / Flex.

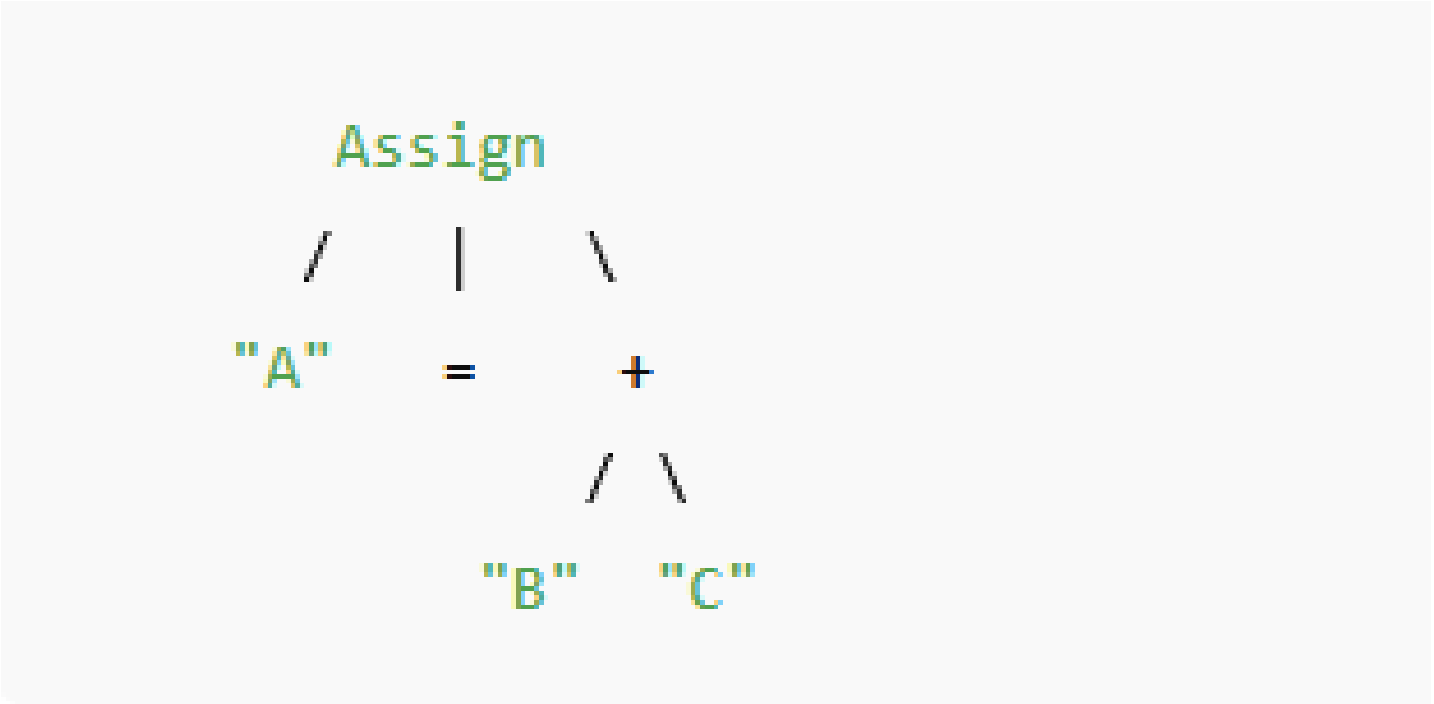
- float area = length * breadth + 2.5;
- Lexemes → Tokens
- float → Keyword
- area → Identifier
- = → Assignment Operator
- length → Identifier
- * → Arithmetic Operator
- breadth → Identifier
- + → Arithmetic Operator
- 2.5 → Floating literal
- ; → Separator

- if (a >= 100) { a = a - 1; }
- Lexemes → Tokens
- if → Keyword
- (→ Left Parenthesis
- a → Identifier
- >= → Relational Operator
- 100 → Integer literal
-) → Right Parenthesis
- { → Left Brace
- a → Identifier
- = → Assignment Operator
- a → Identifier
- - → Arithmetic Operator
- 1 → Integer literal
- ; → Separator
- } → Right Brace

- **2.2 Syntax Analysis**
- Purpose:
- Second phase of compilation; takes tokens from the lexical analyzer and checks if they form a valid syntax according to the grammar.
- Component:
- Parser.
- Responsibilities:
- Detect syntax errors.
- Generate a parse tree or abstract syntax tree (AST).

- Example Parse Tree (for A = B + C;):





1. Full Parse Tree (Concrete Syntax Tree)

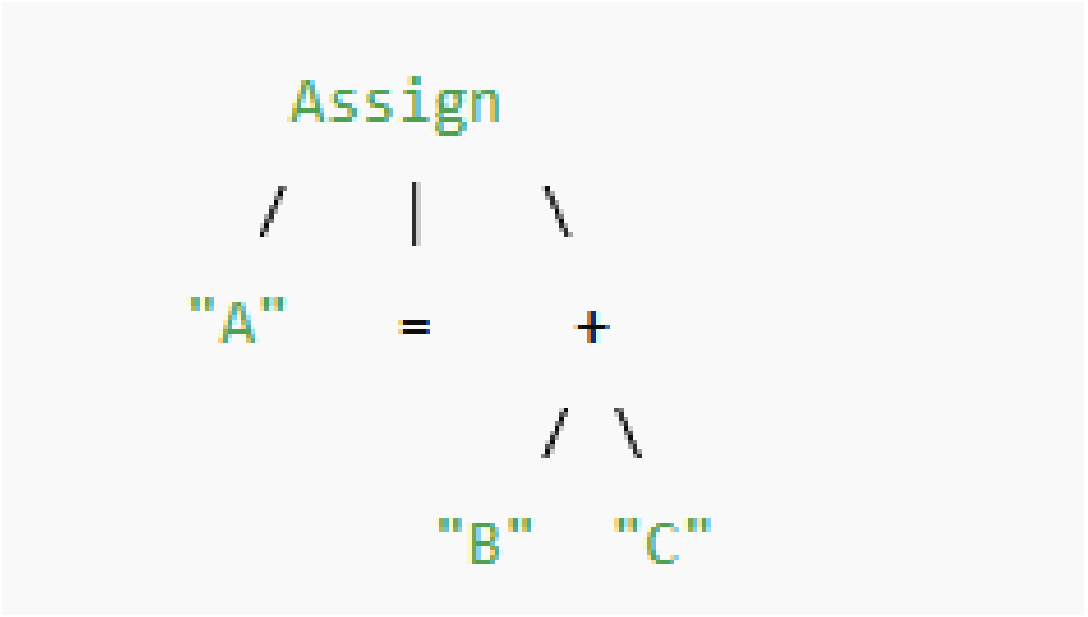
- Here, every nonterminal is expanded according to the grammar:

Each <identifier> is still a nonterminal that could be broken down further by lexical rules:
<identifier> → <letter> <letter-or-digit>*

So, A, B, C could even be split into <letter> nodes in a full parse tree.

2. Abstract Syntax Tree (AST)

- Here, we remove unnecessary grammar details like separate nonterminal expansions and focus on structure:
- Leaves are final tokens from lexical analysis.
- This is the version used by compilers for semantic analysis.



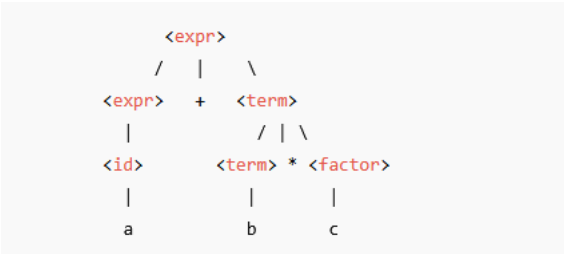
- Example 1: Arithmetic Expression

- Expression:a + b * c

- grammer
- <expr> ::= <expr> "+" <term> | <term>
- <term> ::= <term> "*" <factor> | <factor>
- <factor> ::= <id> | "(" <expr> ")"
- <id> ::= a | b | c

- 1) Parse Tree (Concrete Syntax Tree)

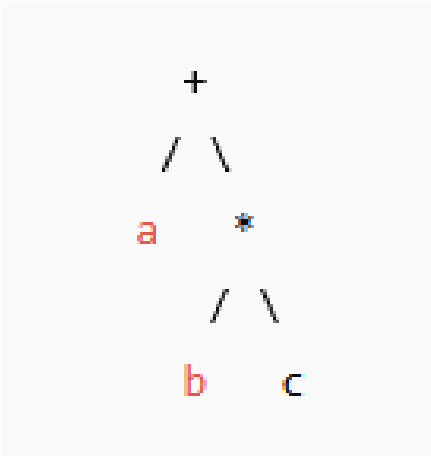
- Follows grammar exactly, showing all rules expanded.



- Here you see all nonterminals (<expr>, <term>, <factor>) explicitly.

2) Abstract Syntax Tree (AST)

- Removes redundant nodes (focus only on operators and operands).
- AST is simpler: only shows essential structure of computation.



Key difference:

- Parse Tree shows every grammar step → might expand A, B, C further.

- AST skips the grammar “noise” and stops at final tokens.

3. Names, Bindings, and Scopes

- 3.1 Names
- Definition:
- A name is a string of characters used to identify a program entity (variable, function, type, etc.).

- Design Issues:

- Length of names (restrictions can reduce readability).

- Case sensitivity (Total vs total in Java are different).

- Special words (keywords, reserved words).

- Reserved words vs Keywords:

- Reserved words: cannot be redefined or used as identifiers.

- Keywords: may have special meaning but can sometimes be reused (less common in modern languages).

- A name is a string of characters used to identify a program entity such as:
- Variable
- Function
- Class / Type
- Constant / Macro
- Module / Package
- Example:
- int total; // "total" is the name of a variable
- float Average; // "Average" is the name of a variable

Design Issues in Names

- When designing a programming language, certain choices affect readability, maintainability, and error-proneness.

1. Length of Names

- Some languages restrict identifier length.
- Too short → less descriptive, harder to read.
- Too long → harder to type, may be truncated in older systems.

- Example:
- In early FORTRAN: Only first 6 characters were significant.

- INTEGER ComputationRate
- INTEGER Computa // Treated as same as ComputationRate in old FORTRAN → bug risk

- Modern languages (C, Java, Python) allow long names:
- total_sales_revenue_for_january = 5000

2. Case Sensitivity

- If a language is case sensitive,
- "Total" ≠ "total" ≠ "TOTAL".
- If a language is case insensitive, these are considered the same name.
- Example:
- Java, C, C++ (case-sensitive):
- int Total = 10;
- int total = 20; // Different variable

- Pascal, SQL (case-insensitive):

- VAR Total: INTEGER;
- VAR total: INTEGER; // Same variable → compiler error

3. Special Words

- Special words give meaning to the compiler.
- Two main types: **Reserved Words and Keywords.**

- **A. Reserved Words**
- Cannot be redefined or used as identifiers.
- Prevents confusion in reading and parsing code.
- Common in modern languages.

- Example:

- In C, int, while, if are reserved words:

- int if = 5; // ❌ Error: "if" is reserved

B. Keywords

- Words that have special meaning in certain contexts but may be reused as identifiers in some older languages.
- Less common in modern languages (most use reserved words for safety).
- Example:
- In Fortran (older versions), INTEGER was a keyword, not reserved:
- INTEGER REAL
- REAL = 10 // Allowed: REAL used as a variable name
- In modern languages, this is almost never allowed.

Reserved vs Keyword Summary Table

Aspect	Reserved Words	Keywords
Can be reused as identifier?	No	Sometimes (older languages)
Purpose	Prevent ambiguity in syntax	Special meaning in certain contexts
Example	if, while, int (Java, C)	REAL in old Fortran

Summary

- Names are fundamental for readability and maintainability.
- Language design must balance flexibility (allowing descriptive names) and safety (avoiding confusion via reserved words).
- Reserved words make parsing easier and safer.
- Keywords are mostly historical; modern languages prefer fully reserved words.

Bindings

- Definition:
- An association between an entity and an attribute (e.g., variable name → memory location).

- Binding Time:
- When this association is created.

- Categories:

- Static binding – occurs before runtime, remains fixed.

- Dynamic binding – occurs during execution, can change.

Attribute	Possible Binding Times
Type of variable	Design time / Compile time
Memory location	Load time / Runtime
Value	Runtime

What is a binding?

- An association between a program entity and one of its attributes, e.g.:
- Name ↔ entity (identifier → variable/function)
- Variable ↔ type (e.g., int x)
- Variable ↔ memory location (address)
- Name ↔ value (initialization/assignment)
- Call site ↔ subprogram body (which procedure/method actually runs)
- Operator symbol ↔ operation (e.g., + meaning integer addition vs string concatenation)

- Storage binding is the association between a program variable (or name) and a specific memory location (storage cell).

- Example:
- int x; // 'x' is bound to a memory location for holding an integer

Storage binding categories

- **Static binding:**
- Storage is bound before execution begins and remains fixed throughout program execution.
- Variables exist for the entire program lifetime.
- Eg: static int a = 10; // allocated once, remains till program ends
- Bound before execution and never change location.
- static int counter = 0; // address fixed for entire run
- **Stack-Dynamic Binding**
- Storage is bound when a function is called (at runtime) and released when the function returns.
- Lifetime: the duration of the function's execution.
- Type fixed at compile time; memory location bound when the block/procedure is entered; released on exit.
- Example:
- void fun() { int b = 5; // stack-dynamic: created at function entry, destroyed at exit}
- void f() {
- int x = 0; // storage bound at call to f(), freed at return
- }

- **Explicit heap-dynamic**

- Objects allocated/deallocated explicitly by the programmer at run time.

- Example: malloc/free in C, new/delete in C++/Java (Java uses GC for deallocation).

- int *p = malloc(sizeof(int)); // bound at runtime
- free(p);

- int* p = (int*) malloc(sizeof(int)); // allocated
- free(p);   // deallocated

- **Implicit heap-dynamic**
- Storage is automatically allocated and deallocated when variables are assigned new values, without explicit programmer control.
- Lifetime: only as long as the variable is referenced.

- Allocation (and often type) bound by assignment/usage, automatically managed.
- Typical in scripting languages (Python, JavaScript).

- x = 42   # x bound to an int object
- x = "hello"   # now bound to a str object (type bound at runtime)

How It Works

- When you assign a value, the runtime system:
- Allocates space on the heap for the value.
- Binds the variable name → heap object.
- Determines the type based on the value (not declared beforehand).
- If later you assign a new value of a different type, the binding changes.
- Old object may be garbage-collected if no references remain.
- New object created on heap and name is rebound.

- x = 42 # Step 1: Heap allocates an int object 42
- # Name 'x' is bound to that object

- x = "hello" # Step 2: Heap allocates a str object "hello"
- # 'x' is rebound to the new object
- # The old int(42) may be garbage collected if unused
- At first, x refers to an integer object (int).

- Later, x is rebound to a string object (str).

- Both allocation and type binding happen dynamically at runtime.

Category	When Bound	Lifetime	Example Languages
Static	Before execution	Entire program execution	C (global/static vars), FORTRAN
Stack-Dynamic	At function entry	Function execution duration	C, Java (local vars)
Explicit Heap-Dynamic	Programmer request	Until explicitly freed	C, C++ (malloc, new)
Implicit Heap-Dynamic	Runtime (on usage)	As long as referenced	Python, JS, Ruby

Static vs Dynamic binding (the big picture)

- **Static binding**
- Doesn’t change during execution.
- Type binding (statically typed languages)
- int add(int a, int b) { return a + b; } // types fixed at compile time
- Overload resolution (compile-time)
- void f(int); void f(double);
- f(10); // picks f(int) at compile time
- Non-virtual method calls in C++/final methods in Java
- class A { static void g() {} } // static methods: target chosen statically
- Pros: performance, early error detection, predictability.
- Cons: less flexible, more declarations/boilerplate.

Dynamic binding

- Occurs at run time and can vary during execution.

- **Dynamic dispatch (polymorphism / virtual methods):**

- class A { void speak()
- {
- System.out.println("A");
- }
- }
- class B extends A
- {
- void speak()
- {
- System.out.println("B");
- }
- }

- A ref = new B();
- ref.speak(); // decides at runtime → prints "B"

- Here, the call site ↔ method body binding is determined by the runtime type of ref.

- **Dynamic type/value binding (implicit heap-dynamic vars):**

- `x = 3.14  # type bound to float at runtime`
- `x = "pi"  # rebound to str at runtime`

- Pros: flexibility, extensibility (plugins, late loading), polymorphism.
- Cons: potential runtime errors, overhead, harder reasoning about performance.